
Evaluating vLLM for Efficient LLM Serving

Vinicius Vieira Resende

College of Engineering and Computer Science
University of Central Florida
Orlando, FL 32816
vi661977@ucf.edu

Abstract

Large language model serving is often limited not only by model size or compute speed, but also by the efficiency of memory management during auto-regressive generation. In particular, the key-value cache can grow substantially with sequence length and the number of active requests, creating memory pressure that can reduce batching efficiency and increase serving latency. This project evaluates the practical serving behavior of vLLM, a system based on PagedAttention proposed by Kwon et al. in “Efficient Memory Management for Large Language Model Serving with PagedAttention” (SOSP 2023). We compare vLLM against a Hugging Face Transformers baseline under controlled changes in batch size, prompt sequence length, and concurrent request rate. By contrasting direct batched generation with vLLM’s paged KV-cache management and dynamic batching, this project analyzes the impact of memory-aware serving on end-to-end LLM inference performance.

1 Introduction

Large language models have become central to many natural language processing applications, but serving them efficiently remains a significant system challenge. Unlike offline model evaluation, interactive LLM serving must support many user requests with low latency while maintaining highly efficient hardware utilization. Auto-regressive generation is especially difficult to optimize because tokens are produced sequentially, and each generated token depends on the previous context. As a result, inference performance is affected not only by the size of the model but also by how efficiently the serving system manages available memory and schedules requests.

A major source of memory pressure during LLM inference is the key-value cache. During generation, transformer models store the keys and values from previous tokens so that future tokens can attend to earlier context without recomputing the entire sequence. While this improves computational efficiency, the KV cache can consume a large amount of GPU memory as prompt lengths, output lengths, and the number of concurrent requests increase. Inefficient KV-cache allocation can lead to memory fragmentation, reduce the number of requests that can be batched together, and ultimately lower throughput.

Kwon et al. address this problem in “Efficient Memory Management for Large Language Model Serving with PagedAttention” by introducing vLLM, a serving system built around the PagedAttention mechanism. Inspired by virtual memory paging, PagedAttention stores KV-cache blocks in non-contiguous memory and maps them logically to each request. This approach reduces memory waste and enables more flexible batching of concurrent requests, improving serving throughput compared to systems that rely on more rigid memory allocation strategies.

In this project, we evaluate the practical serving benefits of vLLM by benchmarking it against a Hugging Face Transformers baseline. Our evaluation focuses on how the two backends behave as inference workloads become more demanding in terms of memory use, batching, and request scheduling. We provide an empirical comparison between direct generation and serving-oriented

inference under controlled workload settings and, through this comparison, study whether vLLM’s paged KV-cache management and dynamic batching provide measurable advantages for end-to-end LLM inference performance.

2 Related Work

The primary work this project builds on is “Efficient Memory Management for Large Language Model Serving with PagedAttention” by Kwon et al., which introduces vLLM as a serving system for improving LLM inference efficiency. The paper identifies KV-cache memory management as a major bottleneck in LLM serving. In standard transformer inference, the KV cache stores previously computed attention keys and values so that future tokens can attend to earlier context without recomputing the full sequence. However, because requests may have different prompt lengths, output lengths, and completion times, KV-cache memory can become fragmented or over-allocated. This reduces the number of requests that can be served simultaneously and limits throughput.

PagedAttention addresses this issue by applying a paging-inspired memory abstraction to the KV cache. Instead of requiring each request’s KV cache to occupy contiguous memory, vLLM divides the cache into fixed-size blocks and maps those blocks logically to each sequence. This allows memory to be allocated more flexibly as requests grow during generation. By improving KV-cache utilization, the system supports more effective batching and higher throughput under concurrent workloads.

3 Method

This project evaluates LLM serving performance by comparing two inference backends: Hugging Face Transformers utilizing `generate()` and vLLM. Both backends are evaluated using the same model, prompt sets, and workload configurations so that performance differences can be attributed primarily to the inference system rather than to changes in input data.

The evaluation is organized around three workload dimensions: batch size, sequence length, and concurrency. Each experiment varies one dimension while keeping the others fixed. This design isolates the effect of each source of serving pressure. Batch-size experiments measure how performance changes when more prompts are processed together. Sequence-length experiments measure the cost of longer contexts and longer generated outputs. Concurrency experiments measure how each backend responds when multiple requests are submitted at the same time.

3.1 Instruction Sets

We leveraged GPT-5.5 to create the instruction files used in this project. The benchmark uses JSONL files containing general-knowledge prompts generated at target lengths of 128, 256, 512, 1024, and 2048 tokens. Each file contains 100 prompts, and each prompt record includes an identifier, the prompt text, the target token length, and an approximate token count. The final prompt lengths were kept within approximately 5% of their target lengths, allowing each prompt set to represent a distinct sequence-length condition.

The prompts are designed to create realistic inference workloads rather than to evaluate factual accuracy. Instead of short question-answer items, they are written as open-ended explanation prompts that encourage sustained generation. During benchmarking, prompt lengths are retokenized using the model tokenizer so that throughput measurements rely on the tokenization behavior of the evaluated model.

3.2 Setup

The evaluation is divided into three controlled sweeps: batch size, sequence length, and concurrency. Each sweep varies one workload dimension while keeping the others fixed, allowing the results to isolate the effect of that specific factor on inference performance.

For the batch-size sweep, the 512-token prompt set is used with a 512-token output limit. Batch size varies across 1, 2, 4, 8, 16, and 32, while concurrency is fixed at 1. This setting measures how each backend responds when more prompts are processed together in a single generation workload.

For the sequence-length sweep, prompt sets of 128, 256, 512, 1024, and 2048 tokens are used. Batch size and concurrency are both fixed at 1. The output length is matched to the prompt target length, so the 128-token prompt setting uses a 128-token output limit and the 2048-token setting uses a 2048-token output limit. This setting measures the effect of longer contexts and longer generations on throughput and latency.

The concurrency sweep is configured similarly to the batch-size sweep, using the 512-token prompt set and a 512-token output limit. In this case, batch size is fixed at 1 while concurrency varies across 1, 2, 4, 8, 16, and 32. This isolates the effect of simultaneous request pressure rather than grouped prompt processing. For Hugging Face, concurrent requests are simulated using Python threads, while vLLM receives concurrent HTTP requests through its serving interface.

3.3 Data Collection

Each experiment writes results to CSV files for later analysis and figure generation. The main metrics collected are generated tokens per second, requests per second, total runtime, and latency.

For Hugging Face, latency is measured around each `generate()` call. In the batch-size sweep, this corresponds to the latency of one batched generation call. In the concurrency sweep, it corresponds to the latency of each submitted generation call under simulated request pressure. For vLLM, latency is measured around OpenAI-compatible HTTP completion requests, while total throughput is computed using wall-clock time over the full benchmark condition.

The output CSVs also includes auxiliary information such as the backend used, model name, prompt length, batch size, and concurrency level. This format allows the results to be grouped by experiment type and plotted against the appropriate variable for each sweep.

4 Experiments

All experiments were conducted on UCF’s Advanced Research Computing Center within the Newton HPC cluster, a Slurm-managed computing environment, and each benchmark job was allocated one NVIDIA H100 PCIe GPU.

The software environment used PyTorch with CUDA support, Hugging Face Transformers for the baseline, and vLLM for the serving backend.

The primary model used in evaluation was Meta’s Llama 2 7B: `meta-llama/Llama-2-7b-chat-hf`. To keep the comparison consistent, both Hugging Face and vLLM used the same prompt files, output settings, and testing configurations.

While the benchmark also records token-level throughput and latency, the figures in this section focus on requests per second as the primary throughput metric.

4.1 Batch Size Sweep

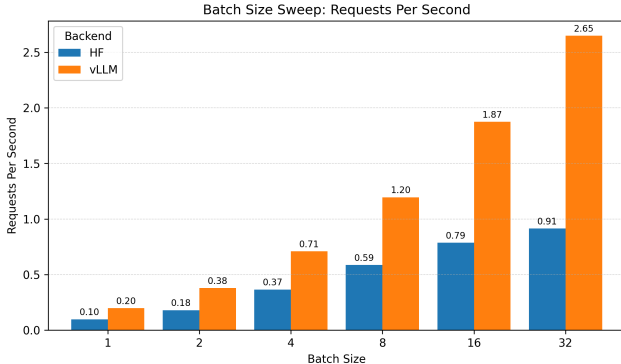


Figure 1: Request throughput comparison between Hugging Face `generate()` and vLLM for the batch-size sweep.

The batch-size sweep measures how each backend responds when the number of prompts processed together increases. As shown in Figure 1, vLLM shows increasing request throughput as batch size grows. This indicates that vLLM is able to benefit from larger grouped workloads through better GPU utilization. Hugging Face also benefits from batching, but its gains are more limited because batching is handled directly through `generate()` rather than through a serving scheduler.

4.2 Sequence Length Sweep

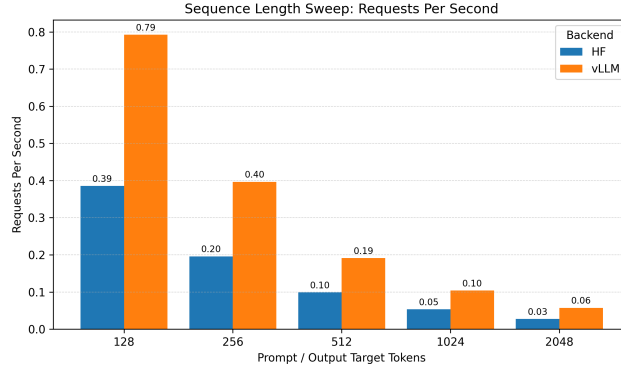


Figure 2: Request throughput comparison between Hugging Face `generate()` and vLLM for the sequence-length sweep.

The sequence-length sweep evaluates the effect of longer prompts and longer generated outputs. Figure 2 shows that request throughput decreases for both backends as sequence length increases. This trend is expected because longer inputs and outputs require more attention computation and increase KV-cache memory usage. Across all tested sequence lengths, vLLM maintains approximately twice the request throughput of the Hugging Face baseline. This suggests that vLLM’s serving-oriented execution and PagedAttention-based KV-cache management provide a consistent efficiency advantage even as the sequence length becomes more demanding.

4.3 Concurrency Sweep

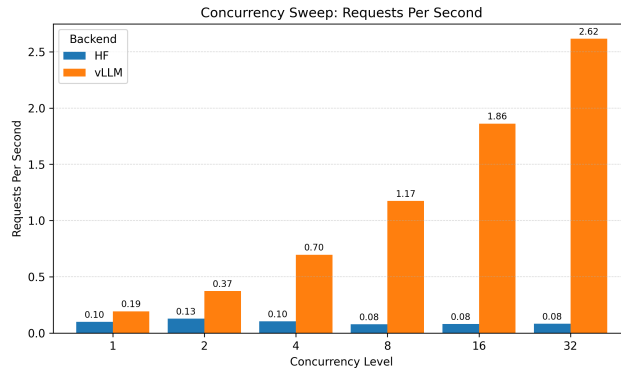


Figure 3: Request throughput comparison between Hugging Face `generate()` and vLLM for the concurrency sweep.

The concurrency sweep measures how each backend responds to simultaneous request pressure. As shown in Figure 3, vLLM scales strongly as concurrency increases, while the Hugging Face baseline remains nearly flat. This is the clearest difference between the two systems. Hugging Face `generate()` is a local generation API, so Python threads can create concurrent calls but do not provide true serving-level scheduling or continuous batching. In contrast, vLLM is designed to

receive concurrent requests and schedule them internally, allowing it to improve throughput as more requests arrive.

5 Analysis & Discussion

The results show a consistent performance gap between direct Hugging Face generation and vLLM serving. Across the measured workloads, vLLM achieves higher request throughput and scales more effectively as inference pressure increases. This supports the central idea behind vLLM: LLM serving performance depends not only on the model and hardware, but also on how the system manages active requests and KV-cache memory during generation.

A key observation is that Hugging Face `generate()` is useful as a direct inference baseline, but it is not designed to behave like a serving engine. In the batch-size setting, Hugging Face can benefit from manually grouping prompts together, but this batching is static and must be specified before generation begins. In the concurrency setting, Python threads can create simultaneous calls, but they do not provide the same request scheduling or continuous batching behavior used in serving systems. This limits the baseline’s ability to take advantage of all available resources under the increasing request pressure.

vLLM, in contrast, is designed specifically for serving workloads. The server can receive requests continuously and schedule them internally, while PagedAttention reduces the system’s memory waste. The observed throughput improvements suggest that these design choices make vLLM better suited for workloads involving larger batches, longer contexts, and concurrent requests. However, they also show that vLLM is not immune to the fundamental computational cost of auto-regressive generation, suggesting that additional optimization strategies may still be useful for long or highly demanding workloads.

6 Conclusion

This project evaluated vLLM against a Hugging Face Transformers baseline under controlled batch-size, sequence-length, and concurrency sweeps. Across the experiments, vLLM achieved higher request throughput and scaled more effectively under increased workload pressure. The results show that PagedAttention-based KV-cache management and serving-oriented scheduling provide practical benefits for LLM inference, especially when handling concurrent requests. Although both systems are affected by the cost of longer generations, vLLM provides a more efficient backend for serving workloads than direct Hugging Face generation.

Acknowledgments

This project employed AI assistants, including ChatGPT and Claude, for code debugging, troubleshooting, and polishing grammar and phrasing in this report. However, the experimental design, methodology, implementation decisions, analysis, and interpretation of results are my own.

References

- [1] Kwon, W. et al. (2023) Efficient Memory Management for Large Language Model Serving with PagedAttention. In *Proceedings of the 29th Symposium on Operating Systems Principles (SOSP '23)*, pp. 611–626. Association for Computing Machinery. DOI: 10.1145/3600006.3613165.
- [2] Touvron, H. et al. (2023) Llama 2: Open Foundation and Fine-Tuned Chat Models. *arXiv preprint arXiv:2307.09288*.
- [3] Hugging Face. (2024) Hugging Face Hub. Model hosting and distribution platform. <https://huggingface.co/>.

A Code Availability

The full project code, prompt files, Slurm scripts, plotting utilities, and README are available at:

<https://github.com/ViniciusVR/vllm-benchmark>

The repository includes instructions for cloning the project, setting up the environment, running the Hugging Face and vLLM benchmark sweeps, and reproducing the figures used in this report.

B Additional Model Results

As an additional scalability check, we repeated the same benchmark sweeps using meta-llama/Llama-2-13b-chat-hf.

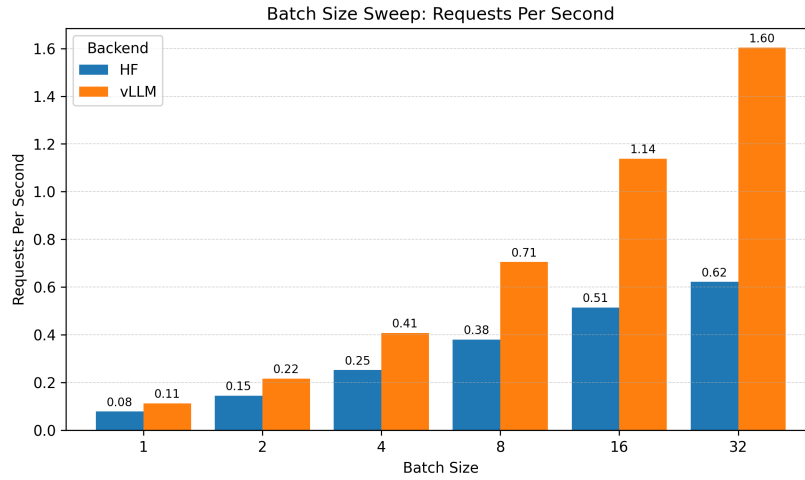


Figure 4: Request throughput comparison for the Llama 2 13B chat model batch-size sweep.

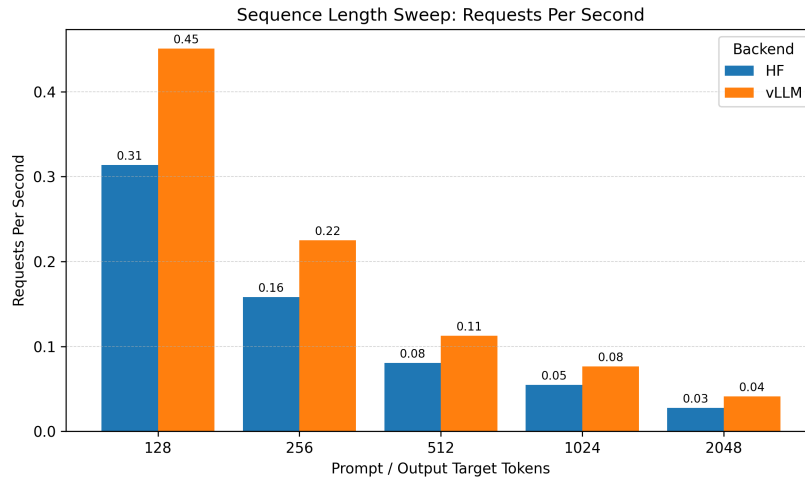


Figure 5: Request throughput comparison for the Llama 2 13B chat model sequence-length sweep.

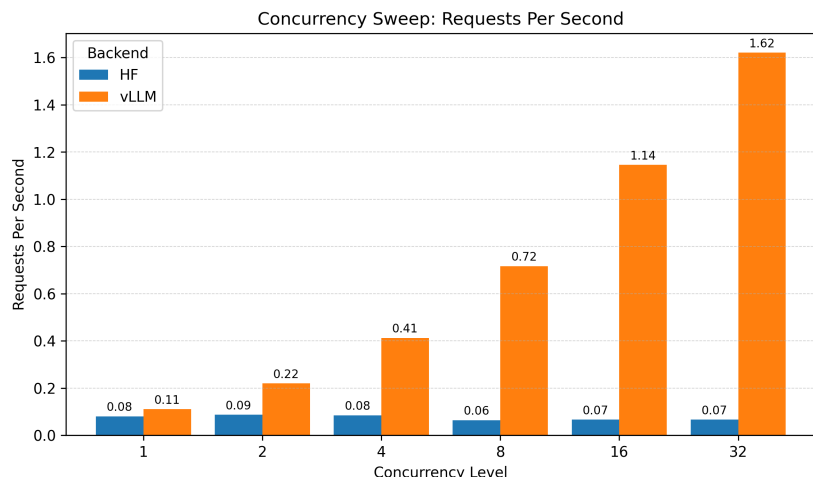


Figure 6: Request throughput comparison for the Llama 2 13B chat model concurrency sweep.

As expected, absolute throughput decreased compared with the 7B model, but the overall trends remained consistent: vLLM achieved higher request throughput than Hugging Face across the batch-size, sequence-length, and concurrency sweeps. Overall, these results suggest that vLLM’s serving-oriented scheduling and PagedAttention-based KV-cache management continue to provide benefits as model size increases, even though larger models reduce absolute throughput for both systems.

C Example Prompt and Model Output

The benchmark prompts were designed as open-ended general-knowledge instructions rather than accuracy-focused question-answer items. One example from the 128-token prompt set is shown below.

Prompt.

Write a general-knowledge explanation about quantum computing for a community organizer preparing a public workshop. The response should be self-contained and should help the reader understand both the basic idea and its broader importance. Where appropriate, include how to distinguish expert consensus from open debate in relation to quantum computing. Make clear which parts of the issue are scientific, which are economic, and which are ethical. Also include a discussion of a real-world analogy that does not oversimplify the issue in relation to quantum computing. In addition, consider how success could be measured using practical indicators in relation to quantum computing.

Generated Output.

Quantum computing is a rapidly advancing field that has the potential to revolutionize the way we approach complex computational problems. At its core, quantum computing is a new way of processing information that leverages the principles of quantum mechanics to perform calculations that are exponentially faster and more powerful than those of classical computers. To understand how this works, it’s important to first understand the limitations of classical computing. Classical computers use bits, which are either a 0 or a 1, to represent information. These bits are processed using logical operations, such as AND and OR, to perform calculations. However, as the complexity